

Código	Tipo	Descrição	Prioridade
RF01	Funcional	O sistema deve exibir na tela inicial ("Início") o estado do canteiro em tempo real, apresentando a umidade atual do solo (%), a temperatura ambiente (°C) e o nível do reservatório de água visualizado através de um tanque animado	Alta
RF02	Funcional	O sistema deve apresentar alertas visuais e recados dinâmicos de orientação baseados no limite de umidade configurado e no volume do reservatório (ex.: "Tudo em ordem", "Solo seco" ou "Reservatório quase vazio")	Alta
RF03	Funcional	O sistema deve permitir a alternância de simulação em tempo real (atualização contínua a cada 4 segundos) e a geração instantânea de uma massa de dados de teste de 30 dias	Média
RF04	Funcional	O sistema deve fornecer uma tela de registros ("Registros") com tabela detalhada exibindo data, hora, temperatura (°C), umidade (%) e reservatório (%), permitindo filtragem por intervalo de datas (De/Até) e faixa de horário	Alta
RF05	Funcional	O sistema deve calcular e exibir o resumo estatístico das leituras filtradas na tela de registros, informando a quantidade de registros, a umidade média e a temperatura média do período	Média
RF06	Funcional	O sistema deve fornecer uma tela de painel analítico ("Painel") com indicadores e gráficos comparativos para períodos de 24 horas, 7 dias e 30 dias, confrontando os dados do período atual com o período anterior de mesma duração	Alta
RF07	Funcional	O sistema deve renderizar gráficos de linha dinâmicos das últimas 24h na tela inicial e gráficos comparativos temporais de umidade e temperatura no painel analítico	Alta
RF08	Funcional	O sistema deve permitir a personalização do perfil do usuário ("Perfil"), cadastrando nome, idade, endereço e o limite percentual de umidade do solo que dispara os alertas do sistema	Média
RF09	Funcional	O sistema deve disponibilizar um botão para apagar o histórico de leituras armazenado no aplicativo e reiniciar os dados locais	Baixa
RF10	Funcional	O sistema deve conter um script em Python (gerador_dados.py) para geração e persistência de dados em banco de dados SQLite (horta.db) e exportação para arquivos CSV (leituras.csv) e Excel (.xlsx)	Média
RNF01	Não funcional	O aplicativo deve ser construído como uma Progressive Web Application (PWA) configurada via `manifest.json`, permitindo instalação na tela inicial do smartphone e execução em modo *standalone*	Alta
RNF02	Não funcional	O sistema deve funcionar de forma completamente offline via Service Worker (service-worker.js), armazenando em cache a casca do app e os arquivos estáticos após o primeiro acesso	Alta
RNF03	Não funcional	O Front-End deve ser desenvolvido utilizando exclusivamente tecnologias web nativas (HTML5, CSS3 e JavaScript Vanilla ES6+), sem dependências de bibliotecas de gráficos ou frameworks externos	Alta
RNF04	Não funcional	A interface do usuário deve ser responsiva e otimizada para dispositivos móveis (Mobile First), contando com suporte nativo a temas claro e escuro (prefers-color-scheme / data-theme) e navegação por barra de abas inferior	Alta
RNF05	Não funcional	Os dados devem ser armazenados localmente no navegador via `localStorage` no objeto desacoplado `Horta.dados`, e o Service Worker deve tratar chamadas da rota `/api/` com a estratégia Rede Primeiro (Network First) para facilitar a integração futura com uma API/backend real	Média
RNF06	Não funcional	O sistema deve carregar todas as páginas e gráficos instantaneamente (em até 2 segundos) e ser compatível com os principais navegadores móveis e desktop modernos (Chrome, Safari, Edge, Firefox)	Alta